# Final Lab: Python Local Setup & GitHub Portfolio

## Lab Goals

**Today's Focus:** Install Python on your computer and create a GitHub portfolio for your final project.

- Install Python locally
- Use the Python REPL (interactive shell)
- Create a GitHub account and repository
- Upload your final project
- Bonus: Introduction to VS Code

## Part 1: Install Python on Your Computer

**Why install Python locally?** Kaggle notebooks require internet access. Installing Python on your computer allows you to write and run code anywhere.

Windows | Mac | Linux

### Windows Installation (Microsoft Store Method)

**Step 1: Open Microsoft Store**

Click the Start menu and search for "Microsoft Store"

**Step 2: Search for Python**

Search for "Python 3.12" or "Python 3.13"

Select the version published by **Python Software Foundation**

**Step 3: Install**

Click "Get" or "Install" and wait for download to complete (5-10 minutes)

**Verify Installation**

Open PowerShell or Command Prompt:

- Press Windows key + R
- Type "cmd" and press Enter

- Type: `python --version`
- You should see: Python 3.12.x or similar

**Alternative: python.org Installer**

If Microsoft Store does not work:

- Go to [python.org/downloads](python.org/downloads)
- Download the Windows installer
- Run the installer
- Check "Add Python to PATH" before installing

**Checkpoint 1: Python is Installed**

Before moving on, confirm you can run `python --version` (Windows) or `python3 --version` (Mac/Linux) and see a version number.

## Part 2: Explore the Python REPL

The REPL (Read-Eval-Print Loop) is Python's interactive mode. It evaluates code immediately, similar to Kaggle notebook cells but in your terminal.

**Step 1: Open the REPL**

**Windows:** Open Command Prompt or PowerShell, type `python`

**Mac/Linux:** Open Terminal, type `python3`

You should see the Python prompt: `>>>`

**Try These Commands in the REPL**

```
>>> print("Hello from my computer")
>>> 5 + 3
>>> name = "Alice"
>>> f"Hello, {name}"
>>> for i in range(5):
...     print(i)
...
>>> numbers = [1, 2, 3, 4, 5]
>>> sum(numbers)
```

**REPL vs Notebooks**

**Similar:** Both execute code interactively

**Different:** REPL is in your terminal, notebooks are in your browser

**Different:** REPL does not save your code automatically

### Step 2: Exit the REPL

Type `exit()` or press Ctrl+D (Mac/Linux) or Ctrl+Z then Enter (Windows)

### Checkpoint 2: REPL Works

You should have successfully opened the REPL, run some Python commands, and exited back to your normal terminal.

## Part 3: Navigate the Command Line

Before running Python scripts, you need to understand basic terminal navigation. These commands work in Command Prompt (Windows), PowerShell (Windows), or Terminal (Mac/Linux).

### Essential Commands

**pwd** (Mac/Linux) or **cd** with no arguments (Windows): Print Working Directory - shows where you are

**ls** (Mac/Linux) or **dir** (Windows): List files and folders in current location

**cd foldername**: Change Directory - move into a folder

**cd ..**: Move up one folder level

### Practice: Where Am I?

Open your terminal and try these commands:

**Windows (Command Prompt or PowerShell):**

```
cd
dir
```

**Mac/Linux (Terminal):**

```
pwd
ls
```

The first command shows your current location. The second lists what is in that location.

### Practice: Navigate to Desktop

**Windows:**

```
cd Desktop
dir
```

**Mac/Linux:**

```
cd Desktop
ls
```

You should now see a list of files and folders on your Desktop.

### Understanding cd (Change Directory)

`cd Desktop` - moves into the Desktop folder

`cd ..` - moves up one level (back to parent folder)

`cd Documents` - moves into Documents folder

Think of folders like a tree. You are always "in" one folder, and you can move up or down.

### Common Mistakes

**Folder does not exist:** If you get an error, the folder name might be wrong. Use `dir` or `ls` to see available folders.

**Spaces in folder names:** If a folder has spaces, use quotes: `cd "My Documents"`

**Case sensitivity:** On Mac/Linux, `Desktop` and `desktop` are different. Use exact capitalization.

## Part 4: Run a Python Script from Terminal

Now that you can navigate folders, you can run Python scripts. Scripts are .py files you execute from the command line.

### Step 1: Create a Python File

Open any text editor (Notepad on Windows, TextEdit on Mac, nano/vim on Linux)

Type this code:

```
print("This is my first script")
numbers = [10, 20, 30, 40, 50]
total = sum(numbers)
print(f"Total: {total}")
```

Save as `test.py` on your Desktop

**Important for Mac users:** In TextEdit, go to Format > Make Plain Text before saving

### Step 2: Navigate to the File

In your terminal, navigate to Desktop where you saved the file:

**Windows:**

```
cd Desktop
```

```
dir
```

**Mac/Linux:**

```
cd Desktop
ls
```

You should see `test.py` in the list of files.

### Step 3: Run the Script

**Windows:**

```
python test.py
```

**Mac/Linux:**

```
python3 test.py
```

You should see the output in your terminal.

### What Just Happened?

1. You navigated to the Desktop folder using `cd`
2. You confirmed the file exists using `dir` or `ls`
3. You ran the Python file using `python` or `python3`
4. Python executed your code and displayed the output

### Checkpoint 3: Script Execution Works

You should have:

- Used `cd` to navigate to Desktop
- Used `dir` or `ls` to verify the file exists
- Created and run a .py file successfully

## Part 4: Create a GitHub Account

GitHub is a platform for hosting code. It allows you to share your projects, collaborate with others, and build a portfolio of your work.

### Step 1: Sign Up

Go to [github.com](https://github.com)

Click "Sign up" in the top right

Enter your email, create a password, and choose a username

Choose a professional username (you may use this for job applications)

**Step 2: Verify Email**

Check your email for verification from GitHub

Click the verification link

**Step 3: Complete Setup**

GitHub will ask some questions about your experience

You can skip these or answer them

**Account Created**

You now have a GitHub account. This will serve as your coding portfolio.

## Part 5: Create Your First Repository

A repository (repo) is like a project folder on GitHub. It contains all your code files and tracks changes over time.

**Step 1: Create New Repository**

Click the "+" icon in the top right corner

Select "New repository"

**Step 2: Configure Repository**

**Repository name:** `csc125-final-project` (or similar)

**Description:** "My Python final project for CSC-125"

**Visibility:** Choose "Public" (so you can share it)

**Initialize:** Check "Add a README file"

Click "Create repository"

**Repository Created**

Your repository is now live at: github.com/YOUR_USERNAME/csc125-final-project

## Part 6: Create Your Project README

You will upload your final project after completing it. For now, create a README that describes what you plan to build.

### Step 1: Edit the README

Click on "README.md" in your repository

Click the pencil icon (Edit) on the right

Replace the content with a description based on your pseudocode document:

```
# CSC-125 Final Project

## Project Description
[Describe what your program will do - refer to your pseudocode]

## Technical Requirements
- Dictionary with at least 3 key-value pairs
- Custom function with parameters/return value
- Loop (for or while)
- Conditional logic (if/elif/else)
- User input with validation
- Clear, meaningful output

## Status
In progress - will upload .ipynb file after completion
```

Click "Commit changes" at the bottom

### Step 2: Upload Your Pseudocode Document

Click "Add file" > "Upload files"

Drag and drop your pseudocode/program design PDF

Add a commit message: "Add program design document"

Click "Commit changes"

### After You Complete Your Final Project

Once you finish coding your project in Kaggle:

1. Download your .ipynb file from Kaggle (make sure all cells are run)
2. Return to your GitHub repository
3. Click "Add file" > "Upload files"
4. Upload your .ipynb file
5. Update your README to reflect project completion

### Checkpoint 4: GitHub Portfolio Created

You should have:

- A GitHub account
- A public repository

- A README describing your planned project
- Your pseudocode PDF uploaded

You will upload your .ipynb file after completing the project.

## Part 7: Share Your Work

### Copy Your Repository URL

Your repository URL is: `https://github.com/YOUR_USERNAME/csc125-final-project`

You can share this link on your resume, LinkedIn, or with potential employers

### REQUIRED: Submit Your Repository URL to Moodle

Copy your full repository URL (example: `https://github.com/YOUR_USERNAME/csc125-final-project`)

Go to Moodle and find **Week 15 Quiz - Python and Github Setup**

Paste your GitHub repository URL in the text submission box

**This is how you will be graded for this lab.**

### What Employers See

When someone visits your GitHub repository, they see:

- Your project description
- Your code (they can read through it)
- When you created/updated the project
- Your commit history (shows your work process)

## Bonus: Introduction to VS Code

VS Code is a popular code editor used by professional developers. It provides features like syntax highlighting, debugging, and extensions. This section is optional but recommended for future development.

### Why Use VS Code?

- Free and works on all operating systems
- Syntax highlighting makes code easier to read
- Built-in terminal
- Git integration (for advanced GitHub workflows)
- Extensions for Python, debugging, and more

### Installing VS Code

### Step 1: Download VS Code

Visit code.visualstudio.com

Download for your operating system

### Step 2: Install

**Windows:** Run the installer, use default settings

**Mac:** Open the .dmg file, drag VS Code to Applications

**Linux:** Follow instructions for your distribution on the VS Code website

### Step 3: Install Python Extension

Open VS Code

Click the Extensions icon on the left sidebar (four squares)

Search for "Python"

Install the extension by Microsoft

## Using VS Code for Python

### Create a Python File

File > New File

Save as `example.py`

Type some Python code:

```
print("Hello from VS Code")
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    print(num * 2)
```

### Run the Code

Right-click in the editor

Select "Run Python File in Terminal"

Or click the play button in the top right

### VS Code Features to Explore

- **IntelliSense:** Auto-completion as you type
- **Syntax Highlighting:** Color-coded keywords and functions
- **Integrated Terminal:** Run commands without leaving the editor
- **Debugging:** Set breakpoints and step through code

**Learning Resources for VS Code**

- Official VS Code Python Tutorial
- Traversy Media - VS Code Setup

## Next Steps

### What You Accomplished Today

- Installed Python on your computer
- Used the Python REPL
- Ran Python scripts from the command line
- Created a GitHub account
- Built your first code portfolio

**Continue Learning**

- **Git/GitHub:** Learn command line Git for better version control
- **Projects:** Build small projects and add them to your GitHub
- **Python Documentation:** docs.python.org
- **Practice:** Work through coding challenges on sites like HackerRank or LeetCode

### Final Notes

You now have the tools to write Python code on your own computer and share your work with the world. Keep building projects, experimenting, and learning. The skills you developed in this course are foundational for any programming career.